Employee Directory

CRUD HR tool · Vue 3 (Composition API) · Axios · Express · MySQL

Mohammed Mohsen Mohammed Alsakkaf
Matric: **A23CS4026**
Web Application Development
Chapter 8 — Individual Assignment

Frontend: Vite @ :5174 | API: Express @ :3001 | DB: MySQL @ :3306 (Laragon) | Driver: mysql2 (pooled, prepared statements) | 3 components + 1 Axios service

1 · Overview

This report documents an **Employee Directory** single-page application that performs full CRUD against a real MySQL database through a Node.js + Express REST API. The browser never touches the database directly — every operation travels Vue → Axios → Express → MySQL — and supports listing, searching, sorting, creating, editing and deleting employees with client- and server-side validation, Ringgit-formatted salaries and a responsive layout. The sections below show the working interface, map each of the four Chapter 8 learning outcomes to specific source files, and discuss the main implementation challenges.

2 · Working User Interface

Figure 1 shows the main screen on page load, listing the seven seeded employees returned by `GET /employees`. The header reports the live *active* headcount.

SCREENSHOT 1

Main screen

Full application: header with the active headcount,
the employee table (7 seeded rows), and the search & sort controls.

`replace this box with <img src="screenshot1.png">`

Figure 1 — Main screen: live active headcount, add/edit form, search & sort controls, and the employee table. Salaries are RM-formatted via `Intl.NumberFormat('ms-MY')`; the inactive row is tinted and badged.

2.1 · Validation & edit states

SCREENSHOT 2

Validation errors

Submit the form with bad input so the inline red error messages are visible.

`replace with <img src="screenshot2.png">`

Figure 2 — Client-side validation: inline errors block submission until every rule passes.

SCREENSHOT 3

Edit mode

Click **Edit** on a row so the record loads into the form (the Employee ID is locked).

`replace with <img src="screenshot3.png">`

Figure 3 — Edit mode: record loaded into the form; the unique `empId` is locked while editing.

3 · Mapping to the Four Chapter 8 Learning Outcomes

LO1 — Connecting Vue to a backend

The front-end has no knowledge of MySQL; it only knows a base URL. The Express layer owns all database access through a connection *pool*, which reuses connections across requests.

`src/api/employeeApi.js`

```
const apiClient = axios.create({
  baseURL: 'http://localhost:3001',
  timeout: 5000,
  headers: { 'Content-Type': 'application/json', Accept: 'application/json' }
})
```

`server/db.js`

```
const pool = mysql.createPool({ host: 'localhost', port: 3306,
  user: 'root', password: '', database: 'employee_directory',
  waitForConnections: true, connectionLimit: 10 })
```

LO2 — The Axios HTTP client

Components never import `axios` directly; they import named CRUD helpers. The request interceptor logs every call, and the response interceptor collapses any server or network failure into one human-readable **`userMessage`** (and surfaces field-level 400 errors back to the form).

`src/api/employeeApi.js — interceptors`

```
apiClient.interceptors.request.use((config) => {
  console.log(`[API] ${config.method?.toUpperCase()} ${config.url}`); return config
})
apiClient.interceptors.response.use((res) => res, (error) => {
  if (error.response?.status === 400 && error.response.data?.errors)
    error.fieldErrors = error.response.data.errors        // → inline form errors
  error.userMessage = /* mapped message */ … ; return Promise.reject(error)
})
```

LO3 — Form handling in Vue

The form is a self-contained child component. It validates every field before emitting, shows inline messages, and refuses to submit while invalid. Modifiers `.trim` and `.number` clean input at the binding level.

`src/components/EmployeeForm.vue — validate() (excerpt)`

```
const empIdRegex = /^EMP[0-9]{3,5}$/
function validate() {
  const e = {}
  if (!empIdRegex.test(f.empId.trim().toUpperCase())) e.empId = 'Format: EMP…'
  if (f.name.trim().length < 3)        e.name   = 'Name must be ≥ 3 chars.'
  if (new Date(f.hireDate) > today)     e.hireDate = 'Cannot be in the future.'
  if (salary < 1500 || salary > 50000) e.salary = 'RM 1,500 – 50,000.'
  errors.value = e; return Object.keys(e).length === 0
}
```

Template binding: `<input v-model.trim="form.empId" />` and `<span v-if="errors.empId" class="err">{{ errors.empId }}</span>`. The parent receives the cleaned payload through `emit('save', …)` and owns the `employees` array (props down, events up).

LO4 — Persistent CRUD with server-side search & sort

Every value reaches MySQL through a `?` placeholder — never string concatenation — so the search box is safe against SQL injection. Sorting is restricted to a whitelist, so the client can never name an arbitrary column.

`server/index.js — GET /employees`

```
let sql = 'SELECT * FROM employees'; const params = []
if (q) { sql += ' WHERE name LIKE ? OR empId LIKE ? OR email LIKE ? OR department LIKE ?'
         params.push(like, like, like, like) }                // 4 columns
const allowedSort = ['name','hireDate','salary','empId']   // whitelist
if (allowedSort.includes(sortBy)) sql += ` ORDER BY ${sortBy} ${dir}`
const [rows] = await pool.query(sql, params)
```

`server/index.js — parameterised INSERT`

```
await pool.query(`INSERT INTO employees
  (empId,name,email,department,position,hireDate,salary,active)
  VALUES (?, ?, ?, ?, ?, ?, ?, ?)`,
  [empId, name, email, department, position, hireDate, salary, active ? 1 : 0])
```

4 · Challenges & How I Resolved Them

- **MySQL returns `DECIMAL` / `DATE` as strings.** Salary arrived as `"8500.00"` and `hireDate` as a full ISO datetime. Fixed by wrapping with `Number()` for RM formatting and slicing `hireDate.slice(0,10)` so the native date input accepts it.
- **Trusting the client is unsafe.** Anyone can call the API with curl, so I added a `validateEmployee()` helper on the server that re-checks the same rules and returns `HTTP 400 { errors }`. The Axios response interceptor copies those field errors back onto the form.
- **SQL injection via the sort parameter.** Column names cannot be parameterised with `?`, so a naive `ORDER BY ${sortBy}` is dangerous. I solved it with a whitelist array and an `includes()` check before interpolation.
- **Duplicate Employee IDs.** The `empId` column is `UNIQUE`; a duplicate insert throws `ER_DUP_ENTRY`. I catch it and return `HTTP 409` with a clear message, and lock the `empId` field during edit so the key cannot drift.
- **Two servers + CORS.** Vite (:5174) and Express (:3001) are different origins, so the browser blocked requests until I added `app.use(cors())`. Keeping the API on :3001 and Vite on :5174 matches the brief and avoids port clashes.